Em Java, um `record` é um tipo especial de classe projetado para servir como um **portador de dados imutável**, reduzindo drasticamente a quantidade de código repetitivo (*boilerplate*). [1, 2, 3]

Foi introduzido de forma preliminar no Java 14 e tornou-se um recurso padrão e estável a partir do **Java 16**. [1, 2]

Principais características

- **Imutabilidade:** Por padrão, todos os campos de um `record` são `private` e `final`. Uma vez que o objeto é criado, os seus valores não podem ser alterados. [1, 2, 3]
- **Geração automática de código:** O compilador cria sozinho:
  - O construtor com todos os argumentos.
  - Os métodos de leitura (no formato `nomeDoCampo()`, e não `getsetNomeDoCampo()`).
  - Os métodos `equals()`, `hashCode()` e `toString()` baseados em todos os componentes. [1, 2, 3]
- **Sintaxe resumida:** Declarar um `record` é muito mais curto do que uma classe comum. [1, 2]

Exemplo de uso

Em vez de escrever uma classe inteira com dezenas de linhas de getters, construtores e comparações, você escreve apenas isto:

java

```
public record Pessoa(String nome, int idade) {}
```

Use o código com cuidado.

Para instanciar e usar:

java

```
Pessoa p = new Pessoa("Ana", 25);
System.out.println(p.nome()); // "Ana"
System.out.println(p.idade()); // 25
System.out.println(p.toString()); // Pessoa[nome=Ana, idade=25]
```

Use o código com cuidado.

Quando usar (e quando não usar)

- **Ideal para:**
  - Objetos de Transferência de Dados (**DTOs**).
  - Respostas de APIs.
  - Chaves de mapas ou agrupamentos temporários de informações. [1, 2, 3]
- **Não recomendado para:**
  - Classes que precisam alterar seus estados internos com `setters`.
  - Entidades de domínio complexas que possuem regras de negócio elaboradas e comportamentos dinâmicos. [1]

Para aprofundar na documentação oficial, consulte o Guia de Records da Oracle. [1]